

Information retrieval: the lexical foundation

Dataset: SciFact (BEIR), 5,183 abstracts

Why these notes exist

Lectures 19–22 sit outside Géron, the course’s primary text. For those four lectures *these notes are the primary source*, and they are examined on exactly the same terms as the textbook parts. Everything in the slide deck for this lecture is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the corpus named above rather than a figure quoted from a paper.

Contents

1	The problem	3
1.1	Why it is not classification	3
1.2	The shape of every search system	3
1.3	An assumption we import	4
1.4	The corpus, counted	4
1.5	The baseline: return the corpus unranked	4
2	From filtering to scoring	4
2.1	First: what is a term?	5
2.1.1	What we are deliberately not doing: stemming	5
2.2	The obvious first idea, and its measurement	5
2.3	Why scoring is nevertheless cheap	6
3	Term weighting, and BM25	7
3.1	Saturation, and the parameter k_1	7
3.2	Inverse document frequency	7

3.3	Length normalisation, and the parameter b	8
3.4	BM25, assembled	9
3.5	One document, scored term by term	9
3.6	BM25, measured	9
3.7	Which correction was doing the work?	10
4	Derivation: evaluating a ranking	10
4.1	What we are given	10
4.2	A worked example, used throughout	11
4.3	Precision and recall, at a cutoff	11
4.4	The cutoff is arbitrary	11
4.5	Reciprocal rank	11
4.6	Average precision	12
4.7	Where AP stops, and what replaces it	12
4.7.1	Why a logarithm, and not something else	12
4.8	Discounted cumulative gain, and its normalisation	13
4.9	One relevant document, moved	14
4.10	Which one to report	14
4.11	Averaging over queries	14
4.12	What none of these metrics measure	15
5	Where the judgements come from	15
5.1	SciFact specifically	15
5.2	Graded relevance changes the answer	16
6	The failure BM25 cannot fix	16
6.1	The ceiling, and what is left on the table	17
7	Reference	17
7.1	Five questions to ask of any retrieval result	17
7.2	Where students lose marks	17
7.3	Vocabulary	18
7.4	Scope	18
7.5	Reading	18
7.6	Summary	18

1 The problem

Eighteen lectures of this course have been about prediction: given an input, produce a label, a number, or a sequence. Retrieval is not that.

The input is a **query** and a **corpus**. The output is an **ordering** of the corpus. Nobody wants a yes or a no; they want the evidence, ranked, so that a person can read the top few and decide.

That change of output changes everything downstream of it. There is no fixed set of classes, because the corpus changes daily. There is no single right answer, because several documents may be relevant and their order matters. And you cannot afford to score the whole corpus with a neural network for every query, because there are millions of documents and the user is waiting.

1.1 Why it is not classification

	Classification	Retrieval
Input	one document	a query and a corpus
Output	a label from a fixed set	an ordering of the corpus
Classes	fixed, known at training	the corpus, changing daily
Right answer	one	several, and their order matters
Cost per prediction	one forward pass	the whole corpus, if you are naive

You could phrase retrieval as classification — one class per document — and it would be useless. The classes change every time a document is added, there are millions of them, and each has a handful of examples.

1.2 The shape of every search system

Every production search system has the same three stages.

1. **Retrieve**: millions → hundreds. Must be cheap.
2. **Re-rank**: hundreds → tens. Can be expensive.
3. **Present**: tens → ten. Diversity, freshness, policy.

This lecture is stage 1, and stage 1 decides the ceiling:

The first-stage ceiling

A document the first stage never returns *cannot* be recovered by any later stage, however good that stage is. This is why we will keep quoting recall@100 for a system whose output is ten results. The first stage is not judged on what the user sees; it is judged on what it makes possible.

Stage 2 is Lecture 20. Stage 3 is mostly not machine learning and mostly not in this course.

1.3 An assumption we import

Whatever we build, a person reads its output, and that person's attention is not uniform: the first result is examined far more often than the second, almost nobody reaches the tenth, and the second page effectively does not exist.

Nothing in this lecture *measures* that. It is an assumption imported from outside, and it is flagged as one here because every metric in Section 4 is an attempt to encode it — and the differences between those metrics are differences of opinion about the shape of the decay.

1.4 The corpus, counted

Quantity	SciFact
Abstracts	5,183
Claims with judgements (test)	300
Mean abstract length, in tokens	225.2
Median abstract length	213
Vocabulary	35,734
Relevant documents per claim, mean	1.13
Claims with exactly one relevant document	92.3%

The last row matters more than it looks. With one relevant document per claim most of the time, recall@10 and the reciprocal rank measure nearly the same thing. On a corpus where every query had twenty relevant documents, they would not, and half of Section 4 would read differently.

SciFact is used here for one reason: it is small enough that we can score *every* document for *every* query, exactly, with no approximate index in the way. Every number in this lecture and the next is therefore a property of the method rather than of an index setting nobody inspected.

1.5 The baseline: return the corpus unranked

Before any method, ask what *no* method scores. Return every document in its stored corpus order, the same order for every query:

Metric	Fixed order
Recall@10	0.0033
Reciprocal rank	0.0017
NDCG@10	0.0012

Effectively zero — and that is the point. Unlike accuracy under imbalance in Lecture 3, a ranking metric has an honest floor near zero. So “better than nothing” is never an argument in retrieval: the comparison that matters is always against the *previous method*, and for the next thirty years that method has been BM25.

2 From filtering to scoring

2.1 First: what is a term?

Before any weighting there is a decision almost nobody writes up: how the text becomes a list of things to count. Take a real claim from the corpus, “0-dimensional biomaterials show inductive properties.” Lower-case it and split on non-alphanumeric characters and you get six tokens:

0 dimensional biomaterials show inductive properties

That is already three decisions, and one of them has done damage: 0-dimensional has become 0 and dimensional, two terms that mean nothing apart. Count how many abstracts contain each:

Term	Abstracts containing it
0	951
dimensional	72
biomaterials	1
show	1,012
inductive	2
properties	211

biomaterials occurs in one abstract of 5,183 and show in 1,012. One of those terms decides the answer and the other decides nothing — and no weighting scheme has been chosen yet. The outcome is already largely determined by the tokeniser.

What we are deliberately not doing: stemming

Our tokeniser treats the following as unrelated terms:

cell 1,837	cells 1,969	cellular 514	
infect 10	infects 7	infection 337	infected 166

A stemmer would merge each row and pool the evidence: a query for infect would then reach the 337 abstracts containing infection rather than only 10. It also merges things that should not be merged, and the net effect on scores is corpus-dependent and usually small. We leave it out because every unexplained step between the text and the number is somewhere a result can hide — and because the mismatch it half-solves has a much better answer in Lecture 20.

2.2 The obvious first idea, and its measurement

Boolean retrieval returns the documents containing *all* the query terms:

$$\text{result} = \bigcap_{t \in q} \text{postings}(t).$$

It is exact, it is fast, and it is how search worked for thirty years. Before reading on, predict: on a corpus of scientific abstracts, and claims written by scientists, how many abstracts contain every word of a claim?

Failure condition — Boolean retrieval, over all 300 test claims

Claims matching no document at all	98.0%
Mean documents matching every term	0.023
Most any claim matched	2

98.0% of claims return an *empty page*. Not a bad ranking — nothing at all. And the failure is silent, because an empty result set is indistinguishable from “no such document exists”.

The conjunction is brittle: one word of the claim absent from an otherwise perfect abstract, and the abstract is excluded entirely. This is the observation that reorganises the whole field:

Stop filtering; start scoring

Every document gets a number. The user gets the largest ones. A missing term then costs *something* rather than *everything*.

2.3 Why scoring is nevertheless cheap

Scoring every document sounds worse than filtering, not better. Naively, per query on this corpus, you would touch 5,183 documents and read 1.17 million tokens — and this is a tiny corpus. A web index is 10^{10} documents and the user is waiting 200 ms.

But almost every one of those documents shares *no* term with the query and contributes exactly zero to any sensible score. Reading them is the waste, and the fix is a data structure rather than a model.

Definition 2.1 (Inverted index). *For every term, store the list of documents containing it, together with the count. One term’s entry is called its **postings list**.*

```
post = defaultdict(list)
for i, doc in enumerate(docs):
    for term, tf in Counter(doc).items():
        post[term].append((i, tf))
```

Four lines, and it is the entire data structure the field is built on. Scoring then visits only $\bigcup_{t \in q} \text{postings}(t)$ — the documents that can possibly score above zero:

```
s = np.zeros(N)
for t in query:
    for i, tf in post.get(t, []):
        s[i] += weight(t, tf, length[i])
```

Vocabulary (distinct terms)	35,734
Postings (term–document pairs)	633,514
Full matrix, were it dense	185 million cells

The postings occupy 0.3% of the dense matrix. Sparsity here is not an optimisation; it is the reason the field is possible at all. Everything that remains to be decided is the function `weight`.

3 Term weighting, and BM25

Start from the obvious idea: a document mentioning *biomaterials* five times is probably more about biomaterials than one mentioning it once. So score by how often the query terms occur. Two failures arrive immediately.

1. **Common words dominate.** Every abstract contains *of* and *the*, so a query containing them matches everything equally.
2. **Long documents win.** A 900-word abstract contains more of everything than a 100-word one and is not thereby more relevant.

The rest of the method is two corrections, one for each failure, and each has a parameter — so each can be measured rather than argued about.

3.1 Saturation, and the parameter k_1

The first correction: the tenth occurrence of a word should not count ten times the first.

$$\frac{\text{tf}(k_1 + 1)}{\text{tf} + k_1} \rightarrow k_1 + 1 \quad \text{as } \text{tf} \rightarrow \infty.$$

tf	1	2	3	5	10	20
factor	1.000	1.310	1.462	1.610	1.743	1.818

From one to two occurrences the factor rises by 0.310; from ten to twenty it rises by 0.075. The curve is bounded, so no single word can dominate a score by repetition — which is, incidentally, the first anti-spam device in search.

k_1 sets how fast the saturation bites: small k_1 saturates almost immediately, large k_1 approaches raw counting. Measured over 150 claims with b held at 0.4:

k_1	0.5	0.9	1.2	2.0
NDCG@10	0.6883	0.6937	0.6944	0.6926

The spread across a fourfold change in k_1 is 0.0061. That is the honest finding, and it is not the one you would guess from how much has been written about tuning this parameter: on abstracts, k_1 barely matters. It matters on long documents, where a term can occur fifty times. *Report the corpus with the parameter, always.*

3.2 Inverse document frequency

The second correction weights a term by how rare it is:

$$\text{idf}(t) = \log\left(1 + \frac{N - n_t + 0.5}{n_t + 0.5}\right),$$

for N documents of which n_t contain t .

Term	Abstracts	idf
of	5,173	0.0020
the	5,159	0.0047
and	5,158	0.0049

of appears in 5,173 of 5,183 abstracts and earns a weight of 0.0020. *No stop-word list was needed.* The weighting derives the stop words from the corpus it is given.

idf replaces the stop-word list

Classical systems shipped a hand-written list of words to delete. That list is language-specific and must be maintained; it is corpus-blind, so in a corpus about the band *The Who* deleting *the* and *who* destroys the query; and its decision is binary, with nothing between delete and keep.

A weight of 0.0020 is not deletion, it is near-deletion, and it is computed *from this corpus*. If a word is common here, it is discounted here, and nobody maintains anything. This is the same move as learning a representation instead of engineering features — arriving thirty years earlier.

The particular form above is used rather than the older $\log(N/n_t)$ because it stays well-behaved for a term appearing in almost every document. Whichever you use, say which: the two agree closely where n_t is small and diverge exactly where a stop word lives.

3.3 Length normalisation, and the parameter b

$$\text{tf}' = \frac{\text{tf}}{\text{tf} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}} \right)}$$

with $b = 0$ meaning no normalisation, so long documents keep their advantage, and $b = 1$ meaning length is divided out completely.

b	0.0	0.4	0.75	1.0
NDCG@10	0.6861	0.6937	0.6917	0.7011

The spread is again small, and again that *is* the finding: SciFact abstracts are of similar length — 225 tokens on average — so there is little here for this correction to correct. On web pages, whose lengths span three orders of magnitude, there would be.

3.4 BM25, assembled

BM25

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}(t) \frac{\text{tf}_{t,d} (k_1 + 1)}{\text{tf}_{t,d} + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}$$

- the **idf** factor: rare terms count for more;
- the **saturating tf**: the tenth occurrence adds far less than the second, because k_1 bounds the growth;
- the **length term**: b decides how much of the document's length is divided out.

Every piece is there because a specific failure demanded it. That is why BM25 has survived thirty years of replacement attempts, and it is the baseline every neural retriever is measured against — including the one in Lecture 20.

3.5 One document, scored term by term

The claim “*Incidence rates of cervical cancer have decreased.*” against the 274-token abstract BM25 ranked first for it:

Term	tf	idf	contribution
cervical	3	4.736	6.786
incidence	5	3.238	5.145
rates	2	2.979	3.802
cancer	5	1.902	3.023
have	3	1.008	1.445
of	6	0.002	0.003
Total			20.204

cervical occurs three times and is worth 6.786. of occurs six times — twice as often — and is worth 0.003. That ratio is the whole method working, and this abstract is one of the judged-relevant ones.

3.6 BM25, measured

Metric	Fixed order	BM25
Precision@1	0.0000	0.5433
Recall@10	0.0033	0.7784
Recall@100	0.0117	0.8852
MRR	0.0017	0.6350
NDCG@10	0.0012	0.6611

The right abstract is in the top ten for 77.8% of claims and top of the list for 54.3%. No training, no labels, no gradient — a data structure and two corrections.

Hold on to recall@100: 88.5%. Almost everything relevant is retrieved *somewhere* in the first

hundred. The gap between @10 and @100 is exactly what re-ranking exists to close, and it is half of Lecture 20.

3.7 Which correction was doing the work?

An argument is not a measurement. Remove each correction in turn, full test set:

Scoring function	NDCG@10
raw term frequency	0.0944
no idf	0.5350
no saturation, no length	0.4801
no length normalisation	0.6490
BM25	0.6611

- **idf is most of the method.** Removing it costs 0.1261. Without it, common words drown the rare ones that identify the document.
- **Saturation earns its place.** idf alone on raw counts reaches 0.4801 against the full method's 0.6611 — roughly 0.18 of NDCG.
- **Length normalisation is nearly free here**, worth about 0.012, which is consistent with the b sweep and with abstracts being of similar length.

The transferable part

Not the ordering — that is a fact about SciFact. It is that each term of a formula can be *removed and priced*, and that a component nobody has priced is a component nobody has justified. The same procedure applies to every architecture in Part IV.

Exercise 19.1

Set $b = 0$ in the notebook and re-score; then remove the idf factor entirely and re-score again. Before running either, write down which of the two you expect to cost more, and by how much. One of them barely matters on this corpus, and you should be able to say which and why from the corpus statistics alone.

4 Derivation: evaluating a ranking

4.1 What we are given

A derivation needs its inputs named. For one query we have

- a ranked list $d_1, d_2, \dots, d_n$ — the system's output, an ordering of the corpus;
- a set R of relevant documents — the judgements, made by people, and treated as truth for now.

Everything that follows is a function of those two objects and nothing else. No scores, no probabilities, no model — which is exactly why the same metrics apply to BM25 here and to a transformer in Lecture 20.

The one assumption is that R is correct and complete. It is neither, and Section 5 is about what that costs.

4.2 A worked example, used throughout

The claim “*Incidence rates of cervical cancer have decreased.*” has three relevant abstracts in the corpus. BM25 placed them at ranks 1, 2 and 8:

✓	✓	.	.	.	.	.	✓	.	.
1	2	3	4	5	6	7	8	9	10

How this example was chosen: it is the first test claim for which BM25 puts at least three relevant abstracts in the top ten, chosen so that the arithmetic is visible — a single hit gives a one-term sum and demonstrates nothing about a discount. It is therefore better than typical, and the honest summary of the method remains the mean over all 300 claims.

4.3 Precision and recall, at a cutoff

$$P@k = \frac{|R \cap \text{top-}k|}{k}, \quad R@k = \frac{|R \cap \text{top-}k|}{|R|}.$$

$P@k$ asks: of what I showed you, how much was good. $R@k$ asks: of what was good, how much did I show you.

On the example with $k = 10$, all three relevant abstracts are inside the top ten, so $R@10 = 3/3 = 1$: perfect recall. And that is the problem. The third abstract sits at rank 8, where a user may never look, and $\text{recall}@10$ cannot say so. It counts a *set*; it does not read an *order*.

4.4 The cutoff is arbitrary

Every fixed k hides the same thing in a different place. Too small, and a method that fills ranks 11–20 with the answer scores zero, identically to one that never finds it. Too large, and rank 1 and rank 100 count the same, so the metric stops rewarding the ordering it exists to measure.

What we actually want is a number that *decreases smoothly* as a relevant document moves down, rather than falling off a cliff at one chosen position. Every metric below is a different answer to that single requirement.

4.5 Reciprocal rank

$$RR = \frac{1}{\text{rank of the first relevant document}}$$

Our example finds one at rank 1, so $RR = 1.0000$. Averaged over queries this is **MRR**, and over all 300 claims BM25 scores 0.6350.

First hit at rank	1	2	5	10
RR	1.0000	0.5000	0.2000	0.1000

Smooth, bounded, and it moves when the order moves. But it stops reading after the first hit: our ranking scores 1.0000 whether the other two abstracts sit at ranks 2 and 8 or nowhere at all. That makes MRR right for “is there a good answer near the top?” and wrong for “did I find the evidence?”.

4.6 Average precision

$$AP = \frac{1}{|R|} \sum_{i: d_i \in R} P@i$$

— the precision at each rank where a relevant document appears.

Hit at rank i	Relevant so far	$P@i$
1	1	1.0000
2	2	1.0000
8	3	0.3750

$$AP = \frac{1}{3} (1.0000 + 1.0000 + 0.3750) = 0.7917.$$

Every hit contributes, and one further down contributes less because the precision above it is worse.

Failure condition — where marks are lost

Divide by $|R|$, not by the number of hits actually found. Dividing by the hit count gives *mean precision*, which is a different and more flattering quantity, and it is the commonest error in this material.

4.7 Where AP stops, and what replaces it

AP assumes relevance is *binary*. Often it is not. An abstract that directly tests a claim and one that mentions it in passing are both “relevant”, and AP cannot prefer the first. Web-search judgements are routinely graded: perfect, excellent, good, fair, bad.

So there are now two requirements: a **gain** that depends on how relevant a document is, and a **discount** that depends on where it sits. Separating those two is the whole idea of the last metric.

Why a logarithm, and not something else

The discount could be any decreasing function. Three candidates, with what each implies about a user:

Discount	Rank 1 vs 2	Rank 9 vs 10	Says
1 (none)	equal	equal	position is irrelevant
$1/i$	halves	−10%	position is nearly everything
$1/\log_2(i+1)$	−37%	−4%	position matters, mildly

No derivation picks one; they encode different beliefs. The logarithm is convention, chosen because it is gentle enough that positions 5 and 10 remain distinguishable while rank 1 still leads. Convention is not truth: if you report NDCG you have adopted this curve, and you should be able to say so out loud.

4.8 Discounted cumulative gain, and its normalisation

$$\text{DCG}@k = \sum_{i=1}^k \frac{g_i}{\log_2(i+1)}$$

Hit at rank i	$\log_2(i+1)$	contribution
1	1.0000	1.0000
2	1.5850	0.6309
8	3.1699	0.3155
DCG@10		1.9464

Is 1.9464 good? The question has no answer, because DCG depends on how many relevant documents exist: a claim with ten relevant abstracts can reach a far larger DCG than one with three, however well both are ranked. So divide by the best this query could possibly have scored — the ideal ranking, all relevant documents first.

Ideal rank	contribution
1	1.0000
2	0.6309
3	0.5000
IDCG@10	2.1309

NDCG

$$\text{NDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} = \frac{1.9464}{2.1309} = 0.9134.$$

The same ranking, under four metrics:

Metric	Value
Recall@10	1.0000
RR	1.0000
AP	0.7917
NDCG@10	0.9134

Recall calls this ranking perfect. It is not: one abstract sits at rank 8. *The metric you report decides what you are allowed to notice.*

4.9 One relevant document, moved

The simplest possible ranking: one relevant document, and the only question is where it sits.

Rank	1	2	5	10
RR	1.0000	0.5000	0.2000	0.1000
NDCG@10	1.0000	0.6309	0.3869	0.2891
Recall@10	1.0000	1.0000	1.0000	1.0000

Recall@10 is constant at 1 across the whole row: it cannot distinguish the best possible ranking from the worst one that still makes the page. RR falls fastest; NDCG falls more gently, by construction. And with 92.3% of our claims having exactly one relevant abstract, *this row is our corpus, most of the time*.

4.10 Which one to report

Question	Metric
Is there a good answer at the top?	MRR
Did I find everything, order aside?	Recall@ k
Did I find everything, order counted?	AP
Graded relevance, position counted?	NDCG@ k
Will the second stage have anything to work with?	Recall@100

The rule

Choose the metric from the *question*, before you see any scores. Choosing it afterwards, from the numbers, is how a paper comes to report recall@100 for the system that lost on NDCG@10 — and it is the single commonest dishonesty in this literature.

4.11 Averaging over queries

One more choice, usually made silently. Given per-query scores, do you average the *scores* or pool the *counts*?

- **Macro:** mean of the per-query metric. Every query counts equally, including one with a single relevant document.
- **Micro:** pool hits and totals across queries first. Queries with many relevant documents dominate.

Every number in these notes is macro: the mean over 300 claims of that claim's own metric. MRR and NDCG are essentially always reported that way; precision and recall sometimes are not, and the two can differ substantially.

The variance across queries is large and almost never reported. A mean NDCG of 0.6611 contains claims scoring 1 and claims scoring 0.

4.12 What none of these metrics measure

- **Latency.** A ranking that takes a minute has a real score of zero and no metric here notices.
- **Redundancy.** Ten identical relevant documents score perfectly and serve the user once.
- **The unjudged.** Anything the pool never saw is counted as irrelevant — Section 5.
- **Harm.** A confidently ranked wrong answer and a merely absent one score identically.

A metric is a choice of what to look at, and it is therefore also a choice of what to ignore.

Exercise 19.2

Take the relevance pattern $\checkmark \cdot \checkmark \cdot \cdot \cdot \cdot \cdot \checkmark$ (relevant at ranks 1, 3 and 10, three relevant documents in total) and compute $P@10$, $R@10$, RR , AP and $NDCG@10$ by hand. Then check each against `rank_metrics` in the notebook. The written paper has asked for exactly this in each of the last three years.

5 Where the judgements come from

Every metric above takes R as given. Somebody had to decide, for 300 claims against 5,183 abstracts, which pairs are relevant. Judging all of them is 1.55 million decisions for this very small corpus. For a web collection it is not large — it is impossible.

Pooling

Run several systems, take the top k from each, judge the union, and treat everything unjudged as *not relevant*. That last clause is an assumption, and it is doing a great deal of work.

Failure condition — what pooling misses

- A document no pooled system ranked highly is unjudged, and therefore counted as irrelevant — whether or not it is.
- So a genuinely new method, which finds documents the pool never saw, is *penalised for succeeding*.
- The pool was built from the systems of its day, so an old collection quietly encodes the assumptions of old methods.

Reported gains on a mature benchmark are partly gains at *matching the pool*. It is why a method can win on the benchmark and disappoint in production, and why a leaderboard difference of a point or two is rarely worth defending.

This is the same shape as the label noise of Lecture 3 and the survivorship of Lecture 2: the ground truth is a measurement, made by someone, under constraints.

5.1 SciFact specifically

SciFact's judgements come from the authors of the claims, who cited the abstracts. They are therefore unusually reliable and unusually *sparse*: 92.3% of claims have exactly one relevant abstract, mean 1.13. An abstract that supports a claim but was not cited is unjudged, and scored as a miss.

Which means our 0.6611 is a *lower bound* on what a reader would call the quality of this ranking, not an estimate of it. Reporting it as “BM25 achieves 0.6611” without that sentence is the kind of claim this course exists to stop.

5.2 Graded relevance changes the answer

SciFact’s judgements are binary. Where they are graded, the gain in DCG is usually taken as $g_i = 2^{\text{rel}_i} - 1$ rather than rel_i itself:

Grade	Linear gain	$2^{\text{rel}} - 1$
0 — bad	0	0
1 — fair	1	1
2 — good	2	3
3 — excellent	3	7

The exponential says an excellent document at rank 3 is worth more than two fair ones at ranks 1 and 2 — a claim about users, not about mathematics, and one you should be prepared to defend when you make it. Two systems can trade places when the gain function changes and nothing else does. State which you used.

6 The failure BM25 cannot fix

The worst single-evidence claim in the test set

*“In mice, *P. chabaudi* parasites are able to proliferate faster early in infection when inoculated at lower numbers than when inoculated at high numbers.”*

BM25 ranks its one relevant abstract **4,999 of 5,183**.

Query terms in the vocabulary	19
Terms the relevant abstract contains	2
Which terms	in, to
Mean idf of the shared terms	0.0378
Mean idf of the missing terms	3.2928

The abstract shares `in` and `to` with the claim it is evidence for. There is nothing for BM25 to match on, and no parameter setting rescues this.

That was the worst case. It is not an isolated one. Over every judged claim–abstract pair in the test set:

Mean fraction of query terms absent from the relevant abstract	40.9%
Pairs sharing fewer than half the query terms	28.6%

A relevant document is missing 40.9% of the query’s words on average. People restate, abbreviate, use the Latin name, or describe the mechanism instead of naming it — and a method whose only evidence is shared strings cannot follow them.

The structural limit

BM25 matches *strings*. Relevance is about *meaning*. That gap is not a tuning problem, and it is exactly what an embedding is for — which is Lecture 20, and the reason Part V comes after Part IV rather than before it.

6.1 The ceiling, and what is left on the table

Metric	BM25	The most it could be
Recall@10	0.7784	1.0000
Recall@100	0.8852	1.0000
NDCG@10	0.6611	1.0000

There are two different opportunities here and they need different methods. Between recall@10 (77.8%) and recall@100 (88.5%) sit documents BM25 found but ranked too low — a *re-ranking* problem, solvable by an expensive model over a hundred candidates. The remaining 11.5% were never retrieved at all; no re-ranker can reach them, and only a first stage that matches on something other than words can. Those are the two halves of Lecture 20.

7 Reference

7.1 Five questions to ask of any retrieval result

1. **Which metric, and at what cutoff?** “Better retrieval” with no metric named is not a claim.
2. **Against which baseline?** If BM25 is absent from the table, ask why. It is free, and it is often close.
3. **Whose judgements, produced how?** Pooled from which systems, and how long ago?
4. **Same corpus, same queries, same preprocessing?** A different tokeniser is a different experiment.
5. **How large is the difference against the spread across queries?** A gain of 0.005 on 300 queries is not a finding.

7.2 Where students lose marks

- Computing DCG but forgetting to divide by the ideal, and then comparing across queries.
- Using $\log_2(i)$ rather than $\log_2(i + 1)$, which divides by zero at rank 1.
- Dividing AP by the number of retrieved hits instead of $|R|$ — that is mean precision, not average precision.
- Quoting a metric with no cutoff. “NDCG of 0.66” is incomplete.
- Describing BM25 as “tf-idf”. It is tf-idf *with the two corrections that make it work*.

7.3 Vocabulary

Inverted index	term $\rightarrow$ the documents containing it, with counts
Postings list	one term's entry in that index
idf	a term's weight, falling as more documents contain it
BM25	saturating tf, length-normalised, idf-weighted
MRR	mean of $1/(\text{rank of first hit})$
NDCG@ k	discounted gain over the best possible discounted gain
qrels	the relevance judgements themselves
Pooling	judging the union of several systems' top results

7.4 Scope

Examinable. The retrieval problem and why it is not classification; the inverted index and why sparsity makes retrieval possible; idf, saturating tf and length normalisation, and BM25 assembled from them; Section 4 in full — $P@k$, $R@k$, RR, AP, DCG, NDCG and which question each answers; pooling, and what unjudged documents do to a reported score.

Not examinable — engineering. Index compression, skip pointers, query parsing, and any particular search library.

Beyond the syllabus. The probabilistic derivation of BM25 from the RSJ model; query expansion and relevance feedback; learning to rank.

7.5 Reading

- These notes are the primary source; the textbook does not cover retrieval.
- Manning, Raghavan and Schütze, *Introduction to Information Retrieval* — chapters 1, 2 and 6 for the index and term weighting, chapter 8 for evaluation. Free online.
- Robertson and Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond* — where BM25 comes from, if you want the derivation we skipped.
- The SciFact and BEIR papers, for how these judgements were made.

7.6 Summary

One line to keep

A ranking is scored by a question, and the question must be chosen before the scores are seen.

Retrieval returns an ordering of a corpus, not a label, and the metrics, baselines and methods all follow from that. The inverted index makes it cheap: postings are 0.3% of the dense matrix. BM25 is three corrections to counting words, each answering a stated failure, and it reaches 0.6611 NDCG@10 with no training at all. A ranking metric answers a question — recall@10 called our worked example perfect while an abstract sat at rank 8. And every one of those numbers rests on judgements someone made on a pool, with everything unjudged counted as irrelevant.